
Comparative Study of Real Bogus Classification Using Attentions

Candidate Numbers: 65013, 62665, 68189, 63593, 68923

Abstract

Astronomical surveys generate millions of candidate detections per night, far more than humans can inspect. Most are bogus: artefacts from imaging noise, optical effects, or errors when subtracting a reference image from a new one. The remainder are real astrophysical events of scientific interest. Separating the two automatically is the real-bogus classification task. We compare a baseline CNN with four attention modules (CBAM, ECA-Net, Coordinate Attention, and SimAM) on the Tomo-e Gozen survey, using a shared backbone, Optuna-tuned hyperparameters and loss, three random seeds per model, and an evaluation pipeline that combines AUROC and AUPRC with temperature scaling, prior correction, and a recall-constrained operating point. ECA-Net gives the highest mean test AUPRC and is the strongest ranking model. Coordinate Attention gives the best operating point after calibration and prior correction, with the highest precision and F1-score at recall above 70%. The choice of attention module therefore depends on whether the goal is ranking or thresholded detection. With only three seeds, the standard deviations are wide enough that we treat these comparisons as tentative.

1 Introduction

Modern astronomical surveys produce far more candidate detections than humans can inspect. Most are bogus detections caused by image artefacts, noise, or imperfect subtraction between a new image and a reference image. A small fraction are real astrophysical transients, such as supernovae or stellar flares. Distinguishing these cases is the real-bogus classification problem. In the Tomo-e Gozen transient survey, around 10^6 bogus detections can appear in one night, so automated filtering is essential [Takahashi et al., 2022].

This task differs from standard natural-image classification. Each Tomo-e Gozen input is a 29×29 cutout with three channels: a science image, a reference image, and their difference image [Takahashi et al., 2022]. These are separate physical image products rather than RGB colour bands. The positive class is also rare: in our test set, only about 0.1% of examples are real transients. A useful model must therefore rank rare positives well while keeping false detections low.

CNN attention modules offer one way to improve feature selection inside convolutional networks. Squeeze-and-Excitation Networks introduced channel attention [Hu et al., 2018], and later modules extended this idea through channel-spatial attention, local cross-channel interaction, coordinate-aware attention, or parameter-free attention weights [Woo et al., 2018, Wang et al., 2020, Hou et al., 2021, Yang et al., 2021]. These methods were developed mainly for general computer vision benchmarks, so it is unclear how they behave when the channels have physical meanings rather than colour semantics.

Prior work on real-bogus classification has used plain CNNs [Cabrera-Vives et al., 2017], Bayesian CNNs [Killestein et al., 2021], CNN-Vision Transformer hybrids [Chen et al., 2023], and task-specific edge-attention modules for supernova detection [Yin et al., 2024]. These studies show that deep image models are effective for transient filtering, but they do not answer our narrower question: how standard CNN attention modules behave on three-channel astronomical cutouts. To

our knowledge, CBAM, ECA-Net, Coordinate Attention, and SimAM have not been compared under a shared backbone, tuning procedure, calibration pipeline, and operating-threshold rule for real-bogus classification.

We compare these four attention modules using the same convolutional backbone, so the main architectural change is the attention block itself. For each architecture, we tune the learning rate, weight decay, dropout rate, module-specific hyperparameters, and loss condition. The loss search includes cross-entropy and focal loss with $\gamma_f \in \{1, 2\}$ [Lin et al., 2017]. The selected configuration is retrained over three random seeds.

Our evaluation separates ranking from thresholded detection. Since the training set has roughly 38% positives while the test set has roughly 0.1%, we apply temperature scaling, correct probabilities for prior shift in odds space, and choose the threshold that maximises precision subject to recall above 70%. In our experiments, ECA-Net gives the highest mean test AUPRC, while Coordinate Attention gives the best recall-constrained operating point after calibration and prior correction.

2 Dataset Description

We use the publicly available real-bogus classification dataset from the Tomo-e Gozen Survey, hosted on Zenodo [Takahashi et al., 2022]. Each sample is a 29×29 pixel cutout comprising three channels: a newly observed image, a past reference image, and their difference image, captured by the Tomo-e Gozen Camera mounted on the Kiso Schmidt Telescope. The task is binary classification of each cutout as either a real astrophysical transient or a bogus artifact.

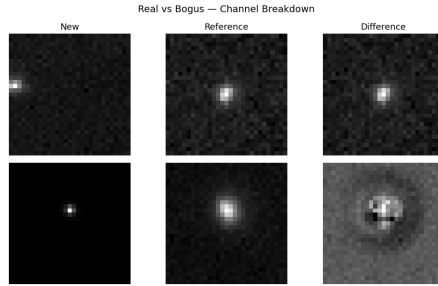


Figure 1: Example real and bogus cutouts across the three image channels.

The full dataset contains 3.26 million samples. Due to computational constraints, we randomly sampled 3% of the data, giving 99,955 examples, and split this subset 85/15 into training and validation sets. The sampled training and validation data have a moderate class imbalance of approximately 62/38 bogus/real. The independently provided test set is kept separate and reflects real operating conditions, with an extreme imbalance of approximately 99.89/0.11 bogus/real. This makes the test set a much harder evaluation of performance under severe class skew.

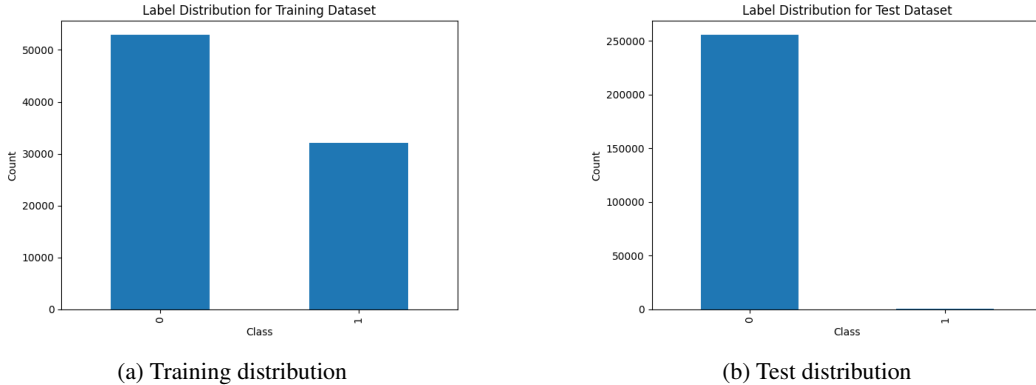


Figure 2: Class distributions for the sampled training data and the held-out test set.

3 Model Architectures

All models use the same convolutional backbone. The baseline CNN defines this backbone, and the attention-based models keep the same feature extractor and classifier while inserting an attention module into the convolutional blocks. This keeps the comparison focused on the attention mechanism rather than changes in the main CNN structure.

3.1 Baseline CNN

The baseline model is a plain convolutional neural network without attention. It provides the reference architecture used by all later variants.

The feature extractor contains three convolutional blocks. Each block applies a 3×3 convolution with padding, followed by batch normalisation, a ReLU activation, and 2×2 max pooling. The channel widths increase from 3 to 16, then to 32, and finally to 64. Adaptive average pooling then reduces the spatial feature map to a single vector. The classifier maps this vector through a fully connected layer with 128 hidden units, a ReLU activation, dropout, and a final linear layer producing two logits for the real and bogus classes.

For an input image $\mathbf{X} \in \mathbb{R}^{3 \times 29 \times 29}$, the baseline model learns a convolutional representation

$$\mathbf{h} = f_{\theta}(\mathbf{X}), \quad (1)$$

where f_{θ} denotes the three-block CNN feature extractor followed by adaptive average pooling and flattening. The classifier then maps this representation to two logits:

$$\mathbf{o} = W_2 \phi(W_1 \mathbf{h} + b_1) + b_2, \quad (2)$$

where ϕ is the ReLU activation. The predicted class probabilities are obtained using the softmax function.

The only architecture-specific hyperparameter tuned for the baseline CNN is the dropout rate. The learning rate, weight decay, and loss function are tuned jointly with dropout using Optuna.

3.2 Convolutional Block Attention Module

Convolutional Block Attention Module (CBAM) [Woo et al., 2018] extends channel attention by using both channel and spatial refinement. Unlike Squeeze-and-Excitation Networks [Hu et al., 2018], which form a channel descriptor using average pooling alone, CBAM uses both average pooling and max pooling. This allows the module to use global channel statistics as well as stronger local activations. CBAM then applies a spatial attention stage to identify where informative regions appear in the feature map.

Let $\mathbf{F} \in \mathbb{R}^{C \times H \times W}$ be an intermediate feature map. CBAM applies two sequential attention refinements:

$$\mathbf{F}' = \mathbf{M}_c(\mathbf{F}) \otimes \mathbf{F}, \quad (3)$$

$$\mathbf{F}'' = \mathbf{M}_s(\mathbf{F}') \otimes \mathbf{F}', \quad (4)$$

where $\otimes$ denotes element-wise multiplication, $\mathbf{M}_c \in \mathbb{R}^{C \times 1 \times 1}$ is the channel attention map, and $\mathbf{M}_s \in \mathbb{R}^{1 \times H \times W}$ is the spatial attention map.

Channel attention. Spatial information is aggregated using average- and max-pooling over the spatial dimensions, giving descriptors $\mathbf{F}_{\text{avg}}^c, \mathbf{F}_{\text{max}}^c \in \mathbb{R}^{C \times 1 \times 1}$. Both descriptors are passed through a shared MLP with bottleneck size C/r , and their outputs are summed:

$$\mathbf{M}_c(\mathbf{F}) = \sigma(\mathbf{W}_1 \mathbf{W}_0 \mathbf{F}_{\text{avg}}^c + \mathbf{W}_1 \mathbf{W}_0 \mathbf{F}_{\text{max}}^c), \quad (5)$$

where $\mathbf{W}_0 \in \mathbb{R}^{C/r \times C}$, $\mathbf{W}_1 \in \mathbb{R}^{C \times C/r}$ are shared weights, ReLU follows $\mathbf{W}_0$, and σ is sigmoid.

Spatial attention. Channel-axis average- and max-pooling of $\mathbf{F}'$ produce two 2D maps $\mathbf{F}_{\text{avg}}^s, \mathbf{F}_{\text{max}}^s \in \mathbb{R}^{1 \times H \times W}$. These are concatenated and convolved:

$$\mathbf{M}_s(\mathbf{F}') = \sigma(f^{7 \times 7}([\mathbf{F}_{\text{avg}}^s; \mathbf{F}_{\text{max}}^s])). \quad (6)$$

The CBAM-specific hyperparameters are the reduction ratio r and the spatial kernel size. The reduction ratio controls the size of the MLP bottleneck, compressing the channel descriptor from C to C/r before projecting it back to C . The spatial kernel size controls the receptive field used to compute the spatial attention map.

3.3 ECA-Net

ECA-Net [Wang et al., 2020] is a lighter channel-attention module. It removes the dimensionality-reduction bottleneck used in SENet [Hu et al., 2018] and avoids the spatial attention branch used in CBAM. Instead, it computes local cross-channel interactions with a one-dimensional convolution over the channel descriptor.

Let $\mathbf{X} \in \mathbb{R}^{C \times H \times W}$ be an intermediate feature map. ECA-Net first aggregates spatial information using global average pooling:

$$z_c = \mathcal{F}_{\text{gap}}(\mathbf{x}_c) = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W x_c(i, j) \quad (7)$$

where $z \in \mathbb{R}^C$ represents one scalar per channel.

Instead of fully dense layers across all channels, ECA-Net applies a 1D convolution over the channel dimension with kernel size k . This captures local interactions between neighbouring channels:

$$\hat{\mathbf{e}} = \text{Conv1D}_k(\mathbf{z}), \quad \mathbf{z} \in \mathbb{R}^C \quad (8)$$

The result is passed through a sigmoid function:

$$\mathbf{e} = \sigma(\hat{\mathbf{e}}) \in (0, 1)^C \quad (9)$$

The learned attention weights are applied channel-wise to the original feature map:

$$\tilde{\mathbf{x}}_c = e_c \cdot \mathbf{x}_c \quad (10)$$

The kernel size is determined from the channel dimension using

$$k = \psi(C) = \left\lfloor \frac{\log_2 C}{\gamma} + \frac{b}{\gamma} \right\rfloor_{\text{odd}} \quad (11)$$

where $\lfloor \cdot \rfloor_{\text{odd}}$ denotes rounding to the nearest odd integer.

The parameters γ and b control how quickly the local channel-interaction width grows with C , and are tuned alongside the optimiser hyperparameters.

3.4 Coordinate Attention

Coordinate Attention (CA) [Hou et al., 2021] modifies channel attention so that positional information is retained during attention computation. Standard SE-style channel attention compresses each feature map into a single scalar using global average pooling, which identifies important channels but discards spatial location. CBAM handles location through a separate spatial attention branch. CA instead embeds position directly into the channel attention mechanism.

CA factorises global pooling into two one-dimensional pooling operations: one along the horizontal direction and one along the vertical direction. For an intermediate feature map $\mathbf{X} \in \mathbb{R}^{C \times H \times W}$, this produces a height-aware descriptor and a width-aware descriptor. These descriptors are concatenated, passed through a shared bottleneck transformation, split back into two branches, and transformed into two attention maps. The input feature map is then reweighted by both height and width attention weights.

Let $\mathbf{X} \in \mathbb{R}^{C \times H \times W}$ be an intermediate feature map. The two coordinate encodings are

$$z_c^h(h) = \frac{1}{W} \sum_{i=0}^{W-1} x_c(h, i), \quad (12)$$

$$z_c^w(w) = \frac{1}{H} \sum_{j=0}^{H-1} x_c(j, w). \quad (13)$$

After concatenation along the spatial axis, the shared transformation is

$$\mathbf{f} = \delta(\text{BN}(\mathbf{W}_1[\mathbf{z}^h, \mathbf{z}^w])), \quad (14)$$

where $\mathbf{W}_1$ is a 1×1 convolution that reduces the channel dimension and δ denotes the h-swish activation [Howard et al., 2019]. The output is split into $\mathbf{f}^h$ and $\mathbf{f}^w$, which are mapped back to C channels and passed through sigmoid functions:

$$\mathbf{g}^h = \sigma(\mathbf{W}_h \mathbf{f}^h), \quad \mathbf{g}^w = \sigma(\mathbf{W}_w \mathbf{f}^w). \quad (15)$$

The refined output is then

$$y_c(i, j) = x_c(i, j) \cdot g_c^h(i) \cdot g_c^w(j). \quad (16)$$

The reduction ratio r is the main CA-specific hyperparameter. It controls the width of the bottleneck and therefore trades representational capacity against parameter cost. Following the original implementation, we use a minimum bottleneck width of eight channels, so the effective hidden dimension is

$$\text{mid} = \max(8, \lfloor C/r \rfloor). \quad (17)$$

With our three convolutional blocks of width 16, 32, and 64, larger values of r keep the bottleneck at this minimum width, while smaller values allow the deepest block to use a wider hidden representation. We tune r over $\{4, 8, 16, 32\}$ alongside the optimiser hyperparameters.

3.5 SimAM

SimAM [Yang et al., 2021] takes a different approach to attention. CBAM, ECA-Net, and Coordinate Attention all introduce additional learnable parameters into the network. SimAM instead derives its attention weights in closed form from the statistics of the feature map itself, and so adds no learnable parameters at all.

The construction is motivated by spatial suppression in the mammalian visual cortex, where a neuron’s importance is determined by how distinct its activity is from that of its neighbours. For each position in a feature map, SimAM defines an energy function that measures how separable that activation is from the others in the same channel. Lower energy corresponds to a more distinctive, and therefore more important, neuron.

Let $\mathbf{X} \in \mathbb{R}^{C \times H \times W}$ be an intermediate feature map. For each neuron t in a channel, the energy function admits a closed-form minimum:

$$e_t^* = \frac{4(\hat{\sigma}^2 + \lambda)}{(t - \hat{\mu})^2 + 2\hat{\sigma}^2 + 2\lambda}, \quad (18)$$

where $\hat{\mu}$ and $\hat{\sigma}^2$ are the mean and variance computed over the spatial dimensions of the channel containing t , and λ is a regularisation coefficient. The feature map is then reweighted neuron-by-neuron using a sigmoid of the inverse energy:

$$\tilde{X}_c(i, j) = X_c(i, j) \cdot \sigma\left(\frac{1}{e_t^*}\right). \quad (19)$$

Because the entire computation depends only on the input feature map and the scalar λ , SimAM produces a full three-dimensional attention weighting across channels, height, and width without changing the network’s parameter count. This contrasts with the other modules in this study, which all factorise attention into one or two of the three axes rather than producing a unified weighting.

The only SimAM-specific hyperparameter is λ . It sets a small floor on the variance term and prevents the attention weights from collapsing on near-uniform feature maps. We tune λ on a logarithmic scale alongside the optimiser hyperparameters.

4 Methodological Framework

4.1 Image Rescaling and Standardisation

Before training, each image was rescaled so that its pixel values lay in the interval $[0, 1]$. This was done separately for each image by shifting the smallest pixel value to zero and scaling the remaining

values relative to the largest value in that image. This placed all inputs on a common numerical scale before channel standardisation.

After rescaling, the mean and standard deviation of each channel were computed using the training set only. The same training-set statistics were then used to standardise the validation and test sets. This keeps preprocessing consistent across all splits while preventing information from the validation or test data entering the training procedure.

4.2 Loss Function Modification

Standard cross-entropy gives the same loss weight to every example. Under class imbalance, this can favour models that learn the majority class well while paying too little attention to the rarer real-transient class. We therefore compared cross-entropy with two focal-loss variants. Focal loss was used because it downweights easy, confidently classified examples and places more emphasis on examples the model finds difficult, which is useful when the positive class is rare.

Oversampling via `WeightedRandomSampler`. For the cross-entropy condition, each training sample was assigned a weight

$$w_i = \frac{1}{n_{y_i}}, \quad (20)$$

where n_{y_i} is the number of training examples belonging to class y_i . Training batches were then drawn with replacement according to these weights. This balances the training signal without changing either the dataset or the loss function. The validation and test loaders were not sampled, so their class distributions remained unchanged.

Focal loss. Focal loss [Lin et al., 2017] modifies cross-entropy by adding the factor $(1 - p_t)^\gamma$:

$$\mathcal{L}_{\text{focal}} = -\alpha(1 - p_t)^\gamma \log(p_t), \quad (21)$$

where p_t is the predicted probability assigned to the correct class, α controls class-level reweighting, and γ is the focusing parameter. When $\gamma = 0$, focal loss reduces to cross-entropy.

The models output two logits, one for each class. To apply PyTorch’s binary `sigmoid_focal_loss`, these were converted into a single binary logit,

$$z = z_1 - z_0, \quad (22)$$

where z_1 and z_0 denote the logits for the real and bogus classes respectively. The focal-loss parameter α was fixed as

$$\alpha = 1 - \bar{y}_{\text{train}}, \quad (23)$$

where $\bar{y}_{\text{train}}$ is the positive-class rate in the training set.

Three loss conditions were compared: cross-entropy with `WeightedRandomSampler`, focal loss with $\gamma = 1$, and focal loss with $\gamma = 2$. The focal-loss conditions used an unweighted training loader, since combining focal reweighting with oversampling would change the effective class prior seen during optimisation.

4.3 Hyperparameter Tuning

Hyperparameters were tuned using Optuna [Akiba et al., 2019]. For each trial, Optuna selected a set of hyperparameter values, trained the model using those values, and evaluated the trained model on the validation set. It then used the validation scores from completed trials to guide the next set of choices. In this study, Optuna used its tree-structured Parzen estimator sampler, which models which regions of the search space have produced better validation scores and samples later trials more often from those regions.

Validation AUPRC was used as the tuning objective. This was preferred over accuracy because the task is highly imbalanced, and the precision-recall curve is more sensitive to performance on the rare positive class.

The same tuning procedure was used for each candidate architecture. The tuned hyperparameters were the learning rate, weight decay, dropout rate, and any architecture-specific parameters. The search was repeated separately under each of the three loss conditions described above. Each condition was therefore treated as part of the model-selection process rather than as a fixed design choice.

4.4 Configuration Selection and Final Training

For each architecture, the selected configuration was the loss function and hyperparameter setting that achieved the highest validation AUPRC during tuning. This selection was made before test evaluation.

The selected configuration was then retrained over multiple random seeds. The seeds controlled random weight initialisation, data-loader randomness, and other stochastic operations in the training procedure. Repeating training across seeds gives a more stable estimate of performance, since a single run can be affected by favourable or unfavourable random initialisation.

Final training used early stopping based on validation AUPRC. During training, the checkpoint with the best validation AUPRC was retained and used for subsequent evaluation. This kept the model-selection criterion consistent between tuning and final training.

4.5 Test Evaluation

After final training, each retained model checkpoint was evaluated on the held-out test set. The validation set was not used to compute the final test metrics. Its role was to tune hyperparameters, select the loss condition, choose the retained checkpoint during early stopping, and fit the calibration temperature described below.

The evaluation included both ranking metrics and threshold-dependent classification metrics. AUROC and AUPRC were used to assess ranking performance, while accuracy, precision, recall, and F1 score were used to assess performance after converting predicted probabilities into class labels.

4.6 Calibration and Prior Correction

Post-hoc calibration was performed using temperature scaling [Guo et al., 2017]. For a trained model, validation logits were collected and a single scalar temperature T was fitted by minimising validation negative log-likelihood. Here, z denotes the model’s binary logit for the real-transient class, computed as the difference between the real-class and bogus-class logits:

$$z = z_1 - z_0. \quad (24)$$

The calibrated logit was then computed as

$$\tilde{z} = \frac{z}{T}. \quad (25)$$

The fitted temperature was applied to both validation and test logits. Calibration quality was assessed using expected calibration error, computed by grouping predictions into confidence bins and comparing average confidence with empirical accuracy in each bin.

A further prior correction was applied because the class prevalence in the training data differed from the prevalence in the test data. Temperature scaling adjusts probability calibration conditional on the validation distribution, but it does not by itself account for a change in the base rate of real transients.

The correction was applied in odds space. If $\hat{p}$ denotes the calibrated model probability, then the calibrated odds are

$$\widehat{\text{odds}} = \frac{\hat{p}}{1 - \hat{p}}. \quad (26)$$

These odds were adjusted by the ratio between the test prior odds and the training prior odds:

$$\widehat{\text{odds}}_{\text{corrected}} = \widehat{\text{odds}} \times \frac{\pi_{\text{test}} / (1 - \pi_{\text{test}})}{\pi_{\text{train}} / (1 - \pi_{\text{train}})}, \quad (27)$$

where π_{train} and π_{test} are the positive-class rates in the training and test sets. The corrected probability was then recovered as

$$\hat{p}_{\text{corrected}} = \frac{\widehat{\text{odds}}_{\text{corrected}}}{1 + \widehat{\text{odds}}_{\text{corrected}}}. \quad (28)$$

4.7 Operating-Threshold Selection

The final operating threshold was selected after temperature scaling and prior correction. Rather than using a fixed threshold of 0.5, thresholds were swept over the corrected probabilities and evaluated using the precision-recall trade-off.

For practical deployment, the threshold was chosen to maximise precision subject to a minimum recall constraint. The main operating point used a recall requirement of at least 70%. This reflects the need to reduce false detections while still retaining a large share of true real transients.

5 Results

This section follows the training workflow. We first report the Optuna-selected configuration for each architecture. We then compare test performance across three random seeds, before moving to calibration, prior correction, and threshold selection on the representative seed.

5.1 Hyperparameter and Loss Selection

Table 1 reports the configuration selected for each architecture. For each model, Optuna tuning was repeated under cross-entropy, focal loss with $\gamma_f = 1$, and focal loss with $\gamma_f = 2$. The configuration with the highest validation AUPRC was used for final training.

Table 1: Best Optuna configuration selected for each model.

Model	Selected loss	Learning rate	Weight decay	Dropout	Attention-specific parameters
Baseline CNN	Focal loss ($\gamma_f = 2$)	0.00164	4.21×10^{-5}	0.0936	N/A
ECA-Net	Focal loss ($\gamma_f = 1$)	0.00447	0.00122	0.0943	ECA $\gamma = 3.503$, $b = 3$
CBAM	Cross-entropy	0.00414	0.01260	0.5637	Reduction ratio = 8, kernel size = 7
Coordinate Attention	Focal loss ($\gamma_f = 1$)	0.00155	1.53×10^{-5}	0.3645	Reduction = 32
SimAM	Cross-entropy	0.01200	7.07×10^{-5}	0.1091	$e_\lambda = 5.42 \times 10^{-6}$

The selected loss was not consistent across models. The Baseline CNN, ECA-Net, and Coordinate Attention chose focal loss, while CBAM and SimAM chose cross-entropy, so adding attention did not always make focal loss the better fit. Dropout choices also varied widely, from 0.09 for the Baseline CNN to 0.56 for CBAM, suggesting that the regularisation needed depends on the attention module rather than on whether attention is present at all.

5.2 Multi-Seed Test Performance

After hyperparameter selection, each final model was trained across three random seeds. Table 2 reports the mean and standard deviation of the test metrics.

Table 2: Test performance across three random seeds. Results are reported as mean $\pm$ standard deviation.

Model	AUROC	AUPRC	Accuracy	Precision	Recall	F1-score
Baseline CNN	0.9976 ± 0.0005	0.7062 ± 0.0587	0.9805 ± 0.0118	0.0650 ± 0.0296	0.9703 ± 0.0154	0.1207 ± 0.0523
ECA-Net	0.9977 ± 0.0012	0.7727 ± 0.0362	0.9899 ± 0.0036	0.1058 ± 0.0365	0.9623 ± 0.0225	0.1892 ± 0.0578
CBAM	0.9955 ± 0.0032	0.7107 ± 0.1038	0.9899 ± 0.0044	0.1129 ± 0.0555	0.9555 ± 0.0157	0.1988 ± 0.0860
Coordinate Attention	0.9967 ± 0.0005	0.7277 ± 0.0704	0.9920 ± 0.0030	0.1325 ± 0.0535	0.9623 ± 0.0103	0.2303 ± 0.0806
SimAM	0.9954 ± 0.0040	0.7119 ± 0.0623	0.9856 ± 0.0129	0.1124 ± 0.0822	0.9612 ± 0.0120	0.1943 ± 0.1301

ECA-Net had the highest mean test AUPRC at 0.7727, roughly 9.4% above the Baseline CNN’s 0.7062 and the highest of any model. AUPRC matters more than AUROC here because the test set is extremely imbalanced: AUROC was above 0.995 for every model, but AUPRC separates them more clearly. Coordinate Attention was second by mean AUPRC and achieved the highest mean accuracy, precision, and F1-score at the default 0.5 threshold. The Baseline CNN had a high AUROC and the strongest mean recall, but the weakest mean precision and F1-score, so a high AUROC alone did not translate into good default-threshold behaviour.

The standard deviations remain wide given only three seeds. ECA-Net’s AUPRC advantage over the Baseline CNN, at 0.0665, is comparable in size to the Baseline CNN’s standard deviation of 0.0587.

We treat the ranking comparisons as suggestive rather than statistically established and return to this point in Section 5.5.

5.3 Calibration Analysis

Calibration was performed on the seed-42 checkpoint of each model. Table 3 reports ECE before and after temperature scaling, on both the validation and test sets.

Table 3: Calibration results for the representative seed before and after temperature scaling.

Model	Representative seed	Temperature	Validation ECE before	Validation ECE after	Test ECE before	Test ECE after
Baseline CNN	42	0.6238	0.0768	0.0216	0.0566	0.0173
ECA-Net	42	0.7994	0.0208	0.0082	0.0142	0.0060
CBAM	42	1.1173	0.0056	0.0042	0.0024	0.0044
Coordinate Attention	42	0.6629	0.0423	0.0097	0.0306	0.0081
SimAM	42	1.0508	0.0053	0.0054	0.0089	0.0076

The Baseline CNN and Coordinate Attention had the highest pre-scaling test ECE and benefited the most from calibration, with test ECE falling from 0.0566 to 0.0173 and from 0.0306 to 0.0081 respectively. CBAM was already well calibrated before scaling and did not improve further; the small increase from 0.0024 to 0.0044 still leaves it with the lowest post-scaling test ECE overall. SimAM was largely unchanged. The fitted temperatures lay between 0.62 and 1.12, so the miscalibration was modest in all cases.

5.4 Prior-Corrected Threshold Selection

The test set contained 292 positive examples and 255,711 negative examples. The training positive rate was 0.3769, while the test positive rate was approximately 0.0011, giving a prior-shift ratio of about 330. We applied the correction in odds space, so thresholds were chosen on probabilities adjusted to the test prior

After correction, thresholds were swept over the corrected probabilities and the operating point was selected as the threshold with the highest precision while still achieving at least 70% recall. Table 4 reports the selected threshold and the resulting confusion-matrix counts for each model.

Table 4: Operating-point performance after temperature scaling, prior correction, and threshold selection.

Model	Representative seed	Threshold	TP	FP	FN	TN	Precision	Recall	F1-score
Baseline CNN	42	0.01040	207	152	85	255559	0.5766	0.7089	0.6359
ECA-Net	42	0.05173	205	87	87	255624	0.7021	0.7021	0.7021
CBAM	42	0.04184	205	102	87	255609	0.6678	0.7021	0.6845
Coordinate Attention	42	0.02779	205	68	87	255643	0.7509	0.7021	0.7257
SimAM	42	0.06987	206	166	86	255545	0.5538	0.7055	0.6205

Coordinate Attention gave the best operating point: 205 of the 292 positives recovered with only 68 false positives, the highest precision (0.7509), and the highest F1-score (0.7257). ECA-Net came close, with the same number of true positives but 87 false positives. The Baseline CNN and SimAM met the recall target as well but generated more than twice the false positives of Coordinate Attention.

5.5 Overall Comparison

The models tell different stories at different points in the pipeline. ECA-Net is the strongest ranking model, with the highest mean test AUPRC and a low post-calibration test ECE of 0.0060. Coordinate Attention is the strongest model after threshold selection, recovering 205 of 292 positives with only 68 false positives at recall above 70%. CBAM produced the best-calibrated probabilities before any post-hoc adjustment but did not lead on either ranking or thresholded performance. SimAM was competitive in default-threshold F1 but produced too many false positives once the prior shift was corrected. The Baseline CNN remains a useful reference: high AUROC and recall, weaker AUPRC, default-threshold precision, and final thresholded F1.

Two factors plausibly drive the ECA-Net / Coordinate Attention split. ECA-Net’s channel attention has no dimensionality-reduction bottleneck, which preserves the distinct semantics of the science, reference, and difference channels and may explain its ranking advantage when the model has to weigh information across all three. Coordinate Attention adds explicit horizontal and vertical positional encoding, which likely helps under thresholding because real transients are typically centred point sources whose location relative to the cutout matters. We cannot disentangle these effects without further testing, but the results are consistent with both.

A caveat applies to all of these comparisons. With only three seeds per model, the standard deviations on AUPRC and F1 are large enough that the gaps between the attention modules and the baseline could close on a fourth or fifth seed. Calibration and threshold selection were also performed on a single seed, so the operating-point results in Table 4 should be read as the behaviour of one trained instance of each model rather than as expected behaviour across runs. We are most confident in the qualitative pattern: attention helps, and the choice of attention module depends on whether the goal is ranking or thresholded operation.

6 Conclusion

This study compared a baseline CNN with four attention modules for real/bogus classification on Tomo-e Gozen cutouts.

While attention helped, no single module won across all metrics. ECA-Net achieved the highest mean test AUPRC across three seeds and was the strongest ranking model. Coordinate Attention achieved the best operating point after temperature scaling and prior correction, with 205 of 292 true positives and only 68 false positives at recall above 70%. The same model is not the right choice for both ranking and thresholded operation. Two mechanisms may influence the split: ECA-Net’s lack of a channel-attention bottleneck preserves the distinct semantics of the science, reference, and difference channels, and Coordinate Attention’s horizontal and vertical positional encoding helps localise centred point sources at decision time.

The prior shift between training and test sets also turned out to matter as much as the choice of threshold. Even after temperature scaling, the chosen operating points only became reasonable once test probabilities were corrected for the ratio between training and test positive rates, which was about 330 in our setup. A pipeline that calibrated without prior correction would have selected very different thresholds.

Several caveats apply. With three seeds per model, the gap between the best attention module and the baseline on AUPRC is comparable in size to the per-model standard deviation, so the ranking conclusions cannot be called statistically significant on three seeds. Calibration and threshold selection were performed on one seed of each model, so Table 4 reports the behaviour of one trained instance rather than expected behaviour across runs. The most natural next step is to repeat the post-processing analysis across all three seeds and to train on more seeds, so the AUPRC and F1 gaps can be tested for significance. Evaluating on a different survey, captured by a different telescope and camera under different observing conditions, would also check that the ranking generalises beyond the Tomo-e Gozen data used here.

References

- T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.
- G. Cabrera-Vives, I. Reyes, F. Förster, P. A. Estévez, and J.-C. Maureira. Deep-HiTS: Rotation invariant convolutional neural network for transient detection. *The Astrophysical Journal*, 836(1): 97, 2017. doi: 10.3847/1538-4357/836/1/97.
- Z. Chen et al. TransientViT: A novel CNN–vision transformer hybrid real/bogus transient classifier for the Kilodegree Automatic Transient Survey, 2023. URL <https://arxiv.org/abs/2309.09937>.
- C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017. URL <https://proceedings.mlr.press/v70/guo17a.html>.
- Q. Hou, D. Zhou, and J. Feng. Coordinate attention for efficient mobile network design. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13713–13722, 2021. doi: 10.1109/CVPR46437.2021.01350. URL <https://arxiv.org/abs/2103.02907>.
- A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam. Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1314–1324, 2019. doi: 10.1109/ICCV.2019.00140. URL <https://arxiv.org/abs/1905.02244>.
- J. Hu, L. Shen, and G. Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7132–7141, 2018. doi: 10.1109/CVPR.2018.00745. URL <https://arxiv.org/abs/1709.01507>.
- T. L. Killestein, J. Lyman, D. Steeghs, K. Ackley, M. J. Dyer, K. Ulaczyk, et al. Transient-optimised real-bogus classification with Bayesian convolutional neural networks: sifting the GOTO candidate stream. *Monthly Notices of the Royal Astronomical Society*, 503(4):4838–4854, 2021. doi: 10.1093/mnras/stab633.
- T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017. doi: 10.1109/ICCV.2017.324. URL <https://arxiv.org/abs/1708.02002>.
- I. Takahashi, R. Hamasaki, N. Ueda, M. Tanaka, N. Tominaga, S. Sako, R. Ohsawa, and N. Yoshida. Deep-learning real/bogus classification for the Tomo-e Gozen transient survey. *Publications of the Astronomical Society of Japan*, 74(4):946–960, Aug. 2022. ISSN 0004-6264. doi: 10.1093/pasj/psac047. URL <https://doi.org/10.1093/pasj/psac047>.
- Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu. ECA-Net: Efficient channel attention for deep convolutional neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11534–11542, 2020. doi: 10.1109/CVPR42600.2020.01155. URL <https://arxiv.org/abs/1910.03151>.
- S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon. CBAM: Convolutional block attention module. In *Computer Vision – ECCV 2018*, Lecture Notes in Computer Science, pages 3–19. Springer, 2018. doi: 10.1007/978-3-030-01234-2_1. URL <https://arxiv.org/abs/1807.06521>.
- L. Yang, R.-Y. Zhang, L. Li, and X. Xie. SimAM: A simple, parameter-free attention module for convolutional neural networks. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 11863–11874. PMLR, 2021. URL <https://proceedings.mlr.press/v139/yang21o.html>.
- K. Yin, J. Jia, F. Li, X. Gao, and T. Sun. Harnessing edge-enhanced attention mechanisms for supernova detection in deep learning frameworks. *Astronomy and Computing*, 46:100784, 2024. doi: 10.1016/j.ascom.2023.100784.